

Agentic Coding Plugin Systems and Cross-Provider Portability

Scope, date, and executive findings

This review examines **installable extension packages for coding agents**: packages that can bundle multiple reusable skills and, ideally, hooks, subagents, commands, MCP integrations, scripts, or other resources. It excludes the former ChatGPT plugin ecosystem and treats MCP servers alone as integrations rather than complete agent-plugin systems.

The requested review date, **August 20, 2026**, is fourteen days in the future relative to the current date, **August 6, 2026**. The defensible evidence cutoff for this report is therefore **August 6, 2026**. Features released between August 7 and August 20 are necessarily outside scope.

The central finding is that the November 2025 conclusion that Claude Code was effectively unique is no longer valid. Claude Code remains the most mature reference implementation, but several systems now provide comparable installable bundles:

- **Claude Code, Cursor IDE, GitHub Copilot CLI, and Gemini CLI** have explicit multi-component package formats that can include skills and hooks, with varying support for subagents and MCP.
- **Codex now has a first-class plugin package and marketplace system** capable of bundling multiple Agent Skills, lifecycle hooks, and MCP integrations. Its largest remaining compositional gap is that the documented Codex plugin manifest does not include plugin-shipped subagents or commands.
- **Pi has a powerful package manager** that bundles multiple Agent Skills with executable TypeScript or JavaScript extensions, prompt templates, themes, and dependencies. Hooks and subagents can be implemented in extension code, but they are not first-class declarative package components.
- **OpenCode has a highly programmable plugin API**, including hooks and transformations of agents, commands, skills, and tools. Its newer API is beta and resembles an application extension API more than a declarative Claude-style plugin bundle.
- **Rulesync has become a provider-neutral intermediate representation for individual agent configuration components**, including skills, hooks, subagents, commands, MCP, and rules. It still does **not** define a provider-neutral plugin manifest, package lifecycle, marketplace catalog, or universal installation format.
- **No genuinely provider-neutral, lossless plugin standard exists.** The portable layer is currently a collection of narrower standards—Agent Skills, MCP, and instruction files—combined with host-specific manifests and build-time adapters.

The most consequential interoperability development is not Rulesync but **explicit Claude-plugin compatibility in other hosts**. GitHub Copilot CLI reads Claude-compatible plugin and marketplace locations and documents installation from Anthropic's Claude Code marketplace. Junie CLI accepts both its native marketplace format and `.claude-plugin/marketplace.json`. Codex recognizes the legacy

Claude marketplace location and exports Claude-compatible hook environment variables, although it does not document full Claude manifest compatibility. ¹

Your November 2025 comparison was useful as a historical snapshot, but its conclusions about Codex, Cursor, Copilot, and the uniqueness of Claude Skills are now materially obsolete. ² Your separate marketplace research also identified important structural precedents, but some of its assertions—such as canonical `skill.yaml` or `skill.json` files, broadly enforced compatibility bounds, and uniform semantic-version handling—should not be treated as verified characteristics of the Claude ecosystem. Current Agent Skills use `SKILL.md`, while marketplace and manifest semantics vary by host. ²

Review method and evaluation model

The review prioritized first-party documentation, source repositories maintained by the tool vendor, and official specifications. A system qualified as a plugin-like implementation when it supported at least one of the following:

1. An installable package containing multiple Agent Skills.
2. A package combining skills with hooks, subagents, commands, tools, or MCP.
3. A code extension that can register several such capabilities as one installed unit.
4. A marketplace or package-manager lifecycle for discovering, installing, upgrading, disabling, or removing the unit.

The comparison distinguishes several concepts that are often conflated:

Agent Skill

A progressively loaded folder centered on `SKILL.md`, optionally containing scripts, references, and assets. The open Agent Skills specification defines metadata-first discovery, loading the full instructions on activation, and reading resources only as needed. ³

Plugin or extension package

An installation and distribution boundary containing one or more skills and potentially other component types. A package requires identity, dependency or source information, component discovery, and usually enablement or update behavior.

Hook

A deterministic event handler triggered around tool execution, session events, file changes, prompting, compaction, subagent execution, or other host lifecycle points. Hook schemas and event taxonomies are not standardized across hosts.

Subagent

A specialized agent definition with its own prompt, tools, model selection, context, or execution policy. Although several tools use Markdown with YAML frontmatter, their fields and orchestration semantics differ.

Provider neutrality

This can refer to three increasingly demanding levels:

Portability level	Meaning	Current status
Content portability	Reuse a skill, instruction file, or MCP server unchanged	Substantial support
Component-model portability	Author hooks, agents, commands, and skills once and generate host-specific representations	Partial; Rulesync is the leading example
Package portability	One manifest, package, marketplace entry, and lifecycle that installs natively across hosts	Not available

The ratings below use **full** for a documented, installable multi-component package; **near-full** where a major component is missing or preview-only; **programmable** where code can construct the equivalent but there is no declarative package model; and **component-only** where portable pieces exist without a unified installable bundle.

Comparative landscape

Tool and surface	Package model	Multiple skills	Hooks	Subagents	MCP or tools	Distribution and lifecycle	Assess
Claude Code	<code>.claude-plugin/plugin.json</code> plus component directories	Yes	Yes	Yes	MCP, LSP, monitors, executables	Local loading, private or public marketplaces, versioned installs	Full re-implementation 4
OpenAI Codex	<code>.codex-plugin/plugin.json</code>	Yes	Yes	Not documented as a plugin component	MCP and registered app connections	Local, repository, Git, npm, private marketplace, public universal directory	Near-full package subagent communication documented

Tool and surface	Package model	Multiple skills	Hooks	Subagents	MCP or tools	Distribution and lifecycle	Assess
Cursor IDE	<code>.cursor-plugin/plugin.json</code>	Yes	Yes	Yes	MCP	Local plugins, multi-plugin repositories, public and team distribution	Full in
Cursor CLI	Individual Cursor components are supported, but <code>.cursor-plugin</code> bundle parity is not clearly guaranteed in official plugin documentation	Yes	Components exist; package behavior unclear	Yes	MCP	No authoritative documentation establishing full IDE-plugin installation parity	Partial under as a pl 7
Pi	npm, Git, or local package with <code>package.json</code> <code>pi</code> manifest	Yes	Through executable extensions	Through extension code rather than native definitions	Custom tools through extensions; no built-in MCP	<code>pi install</code> , update, remove, project/user scopes, npm/Git sources, gallery metadata	Strong progra packag 8
OpenCode	Local or npm JavaScript/TypeScript plugin	Can register or transform skills	Yes, in code	Can register or transform agents	Custom tools and runtime integrations	Local/npm loading; V2 API remains beta	Highly progra not a s declar bundle 9
GitHub Copilot CLI	<code>plugin.json</code> , including Claude-compatible manifest locations	Yes	Yes	Yes	MCP and LSP	Marketplace, repository, local path; Claude marketplaces supported	Full, w strong docum Claude compa
Gemini CLI	<code>gemini-extension.json</code> and conventional component directories	Yes	Yes	Preview	MCP	Installable extensions with management commands	Full or subag previe

Tool and surface	Package model	Multiple skills	Hooks	Subagents	MCP or tools	Distribution and lifecycle	Assess
Junie CLI	Native Junie extension or Claude-compatible marketplace entry	Yes	Not listed as a packaged extension component	Yes	MCP	Git, local, or direct marketplace URL; project/user scope; Claude marketplace support	Near-f unusu interop marke 12

This matrix exposes a major change from 2025: **the relevant dividing line is no longer “Claude has skills and everyone else has only prompts or MCP.”** Agent Skills have become a shared substrate, while competition has moved upward into package composition, lifecycle hooks, subagent configuration, distribution, trust, and marketplace compatibility. The Agent Skills specification itself now formalizes the progressive-disclosure behavior that the 2025 report treated as largely Claude-specific. 13

Findings for the priority tools

Codex

Codex has undergone the largest categorical change since the November 2025 review. The current plugin structure requires `.codex-plugin/plugin.json` and can include:

```

plugin/
├── .codex-plugin/
│   └── plugin.json
├── skills/
│   └── skill-name/
│       └── SKILL.md
├── hooks/
│   └── hooks.json
├── .mcp.json
├── .app.json
└── assets/

```

The manifest can point to multiple skills, MCP server definitions, registered MCP connections, and lifecycle hooks. Hook files can also be auto-discovered at `hooks/hooks.json`. Installed hooks are not trusted automatically: Codex requires users to review the current hook definition before non-managed plugin hooks run. 14

Codex marketplaces support local directories, Git sources and subdirectories, pinned refs or SHAs, and npm packages. Local or private catalogs use `.agents/plugins/marketplace.json`; the legacy `.claude-plugin/marketplace.json` location is also recognized. The runtime supplies both Codex-native

`PLUGIN_ROOT` and `PLUGIN_DATA` variables and Claude-compatible `CLAUDE_PLUGIN_ROOT` and `CLAUDE_PLUGIN_DATA` variables to plugin hook commands. ⁵

Your more recent Codex note correctly identified this new architecture and the legacy marketplace compatibility. ¹⁵ Two refinements are now necessary:

First, OpenAI's public plugin directory is described as a **universal directory shared by ChatGPT and Codex**. This is not the old ChatGPT plugin protocol that you intended to exclude; it is the same newer package model being consumed by Codex. For this review, only its Codex execution and packaging characteristics are relevant. ¹⁵

Second, Codex is not yet a complete Claude Code equivalent. The documented manifest fields cover `skills`, `mcpServers`, `apps`, and `hooks`, but not `agents`, `subagents`, or `commands`. The published directory structure likewise omits `agents/` and `commands/`. The reasonable conclusion is therefore not that Codex cannot run subagents in general, but that **plugin-shipped subagent definitions are not part of the documented Codex plugin contract** as of the cutoff. ¹⁶

This yields a clear verdict:

Codex now supports genuine multi-skill plugins with hooks and MCP, but it does not yet document Claude-style “complete agent teams in a plugin.”

Codex's Claude compatibility is also selective. Recognition of the `.claude-plugin/marketplace.json` catalog location and Claude hook environment variables does not establish that an arbitrary `.claude-plugin/plugin.json`, with Claude agents and commands, is a lossless drop-in Codex plugin. A compatibility adapter or parallel `.codex-plugin/plugin.json` remains the safer design.

Cursor and cursor-cli

Cursor's IDE plugin model is now one of the richest declarative systems. A `.cursor-plugin/plugin.json` package can include:

- Persistent rules.
- Multiple Agent Skills.
- Custom agents.
- Agent-executable commands.
- Hooks and hook scripts.
- MCP servers.
- Assets and plugin variables.

Component paths can be declared explicitly or auto-discovered from conventional directories. The hook taxonomy is extensive and includes events around sessions, tool calls, shell execution, MCP execution, file reads and edits, prompts, compaction, subagent start and stop, and agent responses. ¹⁷

Cursor also supports repositories containing several plugins through `.cursor-plugin/marketplace.json`. That is a first-class multi-plugin catalog, not merely a collection of copied rules. ¹⁸

The significant qualification is **surface parity**. Cursor’s official plugin reference says that these distributable bundles “work in the Cursor IDE.” It does not make the equivalent blanket claim for cursor-cli. Cursor separately documents that subagents can operate in the editor, CLI, and Cloud Agents, but support for a component does not prove that installing a `.cursor-plugin` package activates every packaged component in the CLI. ⁷

For cursor-cli, the evidence supports the following narrower conclusions:

Capability	Cursor CLI status
Agent Skills	Supported as a component
Subagents	Officially supported
MCP	Supported
Rules or instructions	Supported through Cursor configuration
Full <code>.cursor-plugin</code> installation and component parity	Not clearly guaranteed by current public documentation
Drop-in Claude marketplace installation	Not documented

Your existing Cursor note reached the right operational conclusion: a private Claude plugin should be treated as source material, with skills, hooks, and MCP reused individually where compatible, or repackaged under Cursor’s plugin format. Running Claude Code inside Cursor preserves Claude’s marketplace lifecycle but does not make the installed plugin a Cursor Agent plugin. ¹⁸

Cursor does provide useful third-party compatibility below the package layer. It can discover Agent Skills in several shared or Claude-compatible locations, and its third-party-hooks option maps a substantial subset of Claude Code hook configurations into Cursor events. However, some Claude events and tool matcher semantics are unsupported, so this is a translation layer rather than full plugin compatibility. ¹⁹

For a bundle intended to work in both Cursor IDE and cursor-cli, the prudent policy is therefore:

1. Keep skills in standard `SKILL.md` form.
2. Test agents, hooks, and MCP independently in the CLI.
3. Treat `.cursor-plugin/plugin.json` as the IDE distribution contract until Cursor explicitly documents CLI parity.
4. Avoid assuming that a successful IDE marketplace installation implies equivalent CLI activation.

Pi

Pi’s abstraction is different but powerful. A Pi package is an npm, Git, or local package that can bundle:

- Extensions.
- Multiple Agent Skills.
- Prompt templates.
- Themes.

- Runtime dependencies.

A `package.json` can declare these resources under the `pi` key, or Pi can discover conventional `extensions/`, `skills/`, `prompts/`, and `themes/` directories. Packages can be installed globally or per project, updated, filtered by resource type, enabled or disabled, and pinned to npm versions or Git refs. Missing project packages can be installed automatically after a project is trusted. ²⁰

Pi implements the open Agent Skills format and discovers shared `.agents/skills/` locations. It can also be configured to load skills directly from Claude Code and Codex skill directories. Skill metadata is discovered at startup, while full instructions and referenced files are loaded only when needed. ²¹

Pi extensions are the functional equivalent of a general plugin API. They can subscribe to lifecycle events, add custom tools and commands, alter UI behavior, maintain state, and orchestrate arbitrary processes. This means a single Pi package can technically provide “hooks” and subagent-like behavior, but those facilities are implemented in executable extension code rather than represented by portable declarative `hooks.json` and `agents/*.md` components. ²²

This distinction matters for your intended use:

- **As a package manager**, Pi is highly capable.
- **As a declarative cross-agent plugin target**, it is weaker than Claude, Cursor, Copilot CLI, or Gemini.
- **As a programmable host for advanced orchestration**, it may be stronger, because an extension has broad control rather than being limited to a fixed schema.
- **As a security boundary**, it is permissive: Pi explicitly warns that extensions execute arbitrary code with full system access and that skills may direct the model to run executables. ²⁰

Pi therefore belongs in a provider-neutral build system as a special target: standard skills can be copied unchanged, while hooks, subagents, MCP bridges, and orchestration would be emitted as generated TypeScript extension code or provided by a reusable runtime adapter.

OpenCode

OpenCode currently has two overlapping extension models.

Its stable documentation describes local or npm JavaScript and TypeScript plugins that subscribe to events such as tool execution, file editing, session changes, permissions, and installation updates. Plugins can modify tool arguments, execute commands, expose tools, interact with the OpenCode client, and run shell operations. ²³

Its newer V2 plugin API exposes structured transformations over agents, commands, integrations, references, skills, tools, and model catalogs. A plugin can turn an agent into a subagent, add commands or typed tools, and intercept runtime operations. Package plugins are published through ordinary JavaScript package metadata, but the V2 API is explicitly beta and consumers are advised to publish compatibility updates when contracts change. ²⁴

OpenCode also independently supports progressively loaded `SKILL.md` skills and per-agent skill permissions. ²⁵

The result is an inversion of the Codex situation:

- Codex has a stable declarative package shape but fewer component types.
- OpenCode can programmatically create or transform almost every relevant component, but lacks a comparably mature declarative plugin manifest and marketplace lifecycle.

An OpenCode npm plugin can therefore bundle a sophisticated agent system in practical terms. It is not, however, a portable plugin definition. The package's behavior is OpenCode-specific application code coupled to a beta API. For cross-provider authoring, OpenCode should be treated as a **compiled executable target**, not the canonical source format.

Interoperability and provider-neutral abstractions

Rulesync's actual position

Your intuition that Rulesync does not offer a provider-neutral plugin definition remains correct, but it needs an important qualification.

Rulesync now supports a broad canonical component model. Its target matrix includes generation or import for rules, MCP configuration, commands, subagents, skills, hooks, permissions, and checks across tools including Claude Code, Codex CLI, Copilot, Cursor, OpenCode, Pi, Goose, Cline, Kilo Code, and others.

26

Rulesync also has explicit **plugin-packaging targets**, but only for:

- `claudecode-plugin`
- `antigravity-plugin`

For a Claude target, it can generate the plugin's MCP configuration, commands, subagents, skills, and hooks into an existing plugin root. It does not create or modify the plugin manifest, marketplace catalog, scripts, assets, or other package metadata. The root must already exist, and `.claude-plugin/plugin.json` remains separately authored. 27

That makes the precise verdict:

Rulesync is now a provider-neutral authoring and transpilation layer for many **components**, but it is not a provider-neutral **plugin packaging system**.

This distinction also explains an apparent contradiction in Rulesync's support table. Rulesync can target standalone Codex subagent configuration, while the Codex plugin manifest does not document plugin-bundled subagents. "Codex supports subagents" and "Codex plugins do not package subagents" can both be true because standalone host configuration and installable plugin composition are separate surfaces.

28

Agent Skills as the common substrate

Agent Skills are the most successful provider-neutral layer. The specification defines:

```
skill-name/  
├─ SKILL.md  
├─ scripts/  
├─ references/  
└─ assets/
```

The only required file is `SKILL.md`, with `name` and `description` metadata. The progressive-disclosure model is standardized: metadata is exposed during discovery, full instructions are loaded when selected, and supporting resources are read or executed only when required. ¹³

This format is natively recognized or deliberately accommodated by Claude Code, Cursor, Codex, Pi, OpenCode, Gemini CLI, Copilot, and Junie. GitHub has also introduced `gh skill`, in public preview, to search, preview, install, update, validate, and publish skills while targeting different agent hosts and installation scopes. ²⁹

`gh skill` is valuable infrastructure, but it distributes **skills**, not complete plugin bundles. It does not normalize:

- Hook event schemas.
- Subagent definitions.
- Package manifests.
- Marketplace catalogs.
- Plugin-scoped variables.
- Dependency installation.
- Permission and trust policy.
- Upgrade semantics for executable plugin code.

Agent Skills should therefore be understood as the portable payload inside a plugin, not as a portable plugin format.

Claude compatibility as an emerging de facto interchange layer

Several vendors are converging on Claude-compatible paths and formats:

- Copilot CLI checks `.claude-plugin/plugin.json` and `.claude-plugin/marketplace.json`, and GitHub explicitly documents adding Anthropic's Claude Code marketplace. ³⁰
- Junie accepts native `.junie-extension/marketplace.json` or Claude `.claude-plugin/marketplace.json` catalogs. ³¹
- Codex recognizes the legacy Claude marketplace location and provides Claude-compatible environment variables to hook scripts. ⁵
- Cursor can ingest Claude-compatible skill locations and translate a subset of Claude hooks, although it does not install Claude marketplaces as Cursor marketplaces. ¹⁹
- Pi can directly include Claude and Codex skill directories. ²¹

This is significant but should not be mistaken for a standard. Compatibility exists at different levels:

Host	Claude skill compatibility	Claude hook compatibility	Claude plugin manifest compatibility	Claude marketplace compatibility
Copilot CLI	Strong	Strong	Documented	Documented
Junie CLI	Strong	Not established as complete	Marketplace translation or ingestion	Documented
Codex	Strong through shared skill conventions	Partial and deliberate	Not documented as complete	Catalog location recognized
Cursor	Strong	Subset mapping	No	No
Pi	Strong	No declarative import	No	No
OpenCode	Strong skill-level compatibility	No plugin import	No	No

The likely direction is that Claude’s directory conventions become a **de facto source format**, while hosts continue to interpret only the component types they support. This is closer to browser compatibility with a shared subset than to a formally specified universal package.

Why a universal plugin format remains difficult

The remaining incompatibilities are semantic rather than merely syntactic.

Hooks differ in event ontology. Claude uses events such as `PreToolUse` and `PostToolUse`; Cursor exposes `preToolUse`, `postToolUse`, file-specific, MCP-specific, Tab, and workspace events; Gemini has its own lifecycle; Pi and OpenCode expose programmatic events. A name mapping cannot always preserve execution timing, input schemas, blocking behavior, or output semantics. ³²

Subagent definitions differ in capability. Claude plugin agents can declare model, effort, turn limits, tools, skills, memory, background behavior, and worktree isolation, while Cursor’s current plugin reference documents a much smaller portable frontmatter surface. Gemini subagents remain preview, and Pi requires programmatic implementation. ³³

Trust models differ. Codex requires review of plugin hooks; Pi extensions receive full system access; OpenCode plugins execute application code; Claude and Cursor provide their own permission and enterprise-control mechanisms. A universal manifest cannot safely claim identical permissions across these environments. ³⁴

Distribution models differ. Claude, Cursor, Codex, Copilot CLI, Gemini, Junie, Pi, and OpenCode use different combinations of marketplace JSON, Git repositories, npm packages, local directories, caches, enablement files, and version resolution.

Consequently, the realistic abstraction is **source-to-source compilation with explicit loss reporting**, not one package interpreted identically by every host.

Conclusions and recommended architecture

The November 2025 landscape has changed from a near-monopoly to a rapidly converging ecosystem. Claude Code is no longer unique in offering installable multi-component agent extensions. It remains the strongest reference because it combines skills, agents, hooks, MCP, LSP, monitors, executable paths, private marketplaces, public distribution, and a mature component reference in one system. ⁴

For your primary tools, the practical ranking is:

Requirement	Best current fit
Complete declarative bundle with skills, hooks, and subagents	Claude Code or Cursor IDE
Strongest Claude marketplace interoperability	GitHub Copilot CLI
Skills, hooks, and MCP in Codex	Native Codex plugin
Portable skills plus highly programmable runtime behavior	Pi package
Deep programmatic customization of host internals	OpenCode V2 plugin
Native plus Claude-compatible marketplace ingestion	Junie CLI
Cross-provider canonical component authoring	Rulesync
Cross-provider installation of skills only	Agent Skills plus <code>gh skill</code>

For a private plugin repository intended to serve Claude Code, Codex, Cursor, cursor-cli, Pi, and OpenCode, a robust design would use a canonical source tree and generated host adapters:

```
agent-suite/  
├─ pack.yaml           # Canonical package metadata  
├─ skills/             # Standard Agent Skills; copied unchanged  
├─ agents/             # Canonical normalized agent definitions  
├─ hooks/              # Abstract events and portable handlers  
├─ commands/  
├─ rules/  
├─ mcp/  
├─ scripts/  
├─ assets/  
├─ adapters/  
│   ├─ claude/  
│   ├─ codex/  
│   ├─ cursor/  
│   └─ pi/
```

```

|   └─ opencode/
└─ generated/
|   └─ claude-plugin/
|   └─ codex-plugin/
|   └─ cursor-plugin/
|   └─ pi-package/
|   └─ opencode-package/
└─ compatibility.lock

```

`pack.yaml` should be an internal source schema, not presented as a universal runtime standard. It would declare components and required capabilities, for example:

```

name: engineering-suite
version: 1.4.0

components:
  skills:
    - skills/code-review
    - skills/release-management

  agents:
    - agents/security-reviewer.yaml
    - agents/test-planner.yaml

  hooks:
    - event: before-tool
      matcher:
        tool-class: shell
      handler: scripts/guard-shell.py
      blocking: true

  mcp:
    - mcp/context7.json

targets:
  claude:
    required: [skills, agents, hooks, mcp]

  codex:
    required: [skills, hooks, mcp]
    unsupported:
      agents: warn

  cursor:
    required: [skills, agents, hooks, mcp]

```

```

pi:
  compile:
    hooks: extension
    agents: extension

opencode:
  compile:
    agents: plugin-api
    hooks: plugin-api

```

The build should produce:

- `.claude-plugin/plugin.json` and `.claude-plugin/marketplace.json`.
- `.codex-plugin/plugin.json` and `.agents/plugins/marketplace.json`.
- `.cursor-plugin/plugin.json` and optionally `.cursor-plugin/marketplace.json`.
- `package.json` with a `pi` resource manifest and generated Pi extensions.
- An OpenCode npm plugin that registers canonical agents, commands, skills, and hooks.
- A machine-readable compatibility report identifying omitted or degraded behavior.

Rulesync can already provide much of the component conversion layer, especially for rules, skills, agents, commands, hooks, MCP, and permissions. It cannot currently own the complete build because it does not emit Codex, Cursor, Pi, or OpenCode plugin packages, and even its Claude packaging target leaves manifests, catalogs, scripts, and assets outside its control. ³⁵

The most sustainable near-term strategy is therefore:

1. **Use Agent Skills as the immutable portable core.**
2. **Use MCP as the portable external-tool boundary.**
3. **Maintain normalized source definitions for agents and hooks.**
4. **Generate separate native plugin packages for each host.**
5. **Treat Claude compatibility as an optimization, not a guarantee.**
6. **Run host-specific contract tests, particularly for hook blocking behavior, path variables, subagent isolation, and CLI-versus-IDE parity.**
7. **Fail builds when a required capability cannot be represented instead of silently dropping it.**

The final answer to the provider-neutral question is consequently two-part:

A provider-neutral definition for skills and many individual configuration components now exists in practice, and Rulesync is a credible implementation of that layer. A provider-neutral definition for an installable, versioned, secure, multi-component agent plugin does not yet exist.

The closest available architecture is not “write one plugin and run it everywhere,” but **author one canonical agent suite and compile it into several native plugins, preserving standard `SKILL.md` directories wherever possible and surfacing semantic losses explicitly.**

1 10 About GitHub Copilot plugins - GitHub Docs

https://docs.github.com/en/copilot/concepts/agents/about-plugins?utm_source=chatgpt.com

2 3 13 Specification - Agent Skills

https://agentskills.io/specification?utm_source=chatgpt.com

4 Create plugins - Claude Code Docs

https://code.claude.com/docs/en/plugins?utm_source=chatgpt.com

5 14 15 16 34 Package your plugin – Plugins | OpenAI Developers

<https://developers.openai.com/codex/plugins/build>

6 7 17 18 cursor.com

<https://cursor.com/docs/reference/plugins.md>

8 20 pi/packages/coding-agent/docs/packages.md at main · earendil-works/pi · GitHub

<https://github.com/badlogic/pi-mono/blob/main/packages/coding-agent/docs/packages.md>

9 23 Plugins | OpenCode

<https://opencode.ai/docs/da/plugins/>

11 gemini-cli/docs/extensions/reference.md at main · google-gemini/gemini-cli · GitHub

https://github.com/google-gemini/gemini-cli/blob/main/docs/extensions/reference.md?utm_source=chatgpt.com

12 31 Add and configure extensions | Junie Documentation

https://junie.jetbrains.com/docs/junie-cli-extensions.html?utm_source=chatgpt.com

19 cursor.com

<https://cursor.com/docs/skills.md>

21 pi/packages/coding-agent/docs/skills.md at main · earendil-works/pi · GitHub

<https://github.com/badlogic/pi-mono/blob/main/packages/coding-agent/docs/skills.md>

22 pi/packages/coding-agent/docs/extensions.md at main · earendil-works/pi · GitHub

<https://github.com/badlogic/pi-mono/blob/main/packages/coding-agent/docs/extensions.md>

24 Plugins - OpenCode

<https://opencode.ai/v2/docs/build/plugins>

25 Agent Skills | OpenCode

<https://opencode.ai/docs/skills>

26 28 Supported Tools and Features | Rulesync

<https://rulesync.dyoshikawa.com/reference/supported-tools.html>

27 35 Plugin Packaging | Rulesync

<https://rulesync.dyoshikawa.com/guide/plugin-packaging.html>

29 Adding agent skills for GitHub Copilot - GitHub Enterprise Cloud Docs

https://docs.github.com/en/enterprise-cloud%40latest/copilot/how-tos/copilot-on-github/customize-copilot/customize-cloud-agent/add-skills?utm_source=chatgpt.com

30 Finding and installing plugins for GitHub Copilot CLI - GitHub Docs

https://docs.github.com/en/copilot/how-tos/copilot-cli/customize-copilot/plugins-finding-installing?utm_source=chatgpt.com

32 Hooks reference - Claude Code Docs

https://code.claude.com/docs/en/hooks?utm_source=chatgpt.com

33 Plugins reference - Claude Code Docs

https://code.claude.com/docs/en/plugins-reference?utm_source=chatgpt.com